

코드 리뷰, 어떻게 해야 할까?

4장. 코드 리뷰 이해하기 / 5장. 코드 리뷰를 잘 하는 방법

Do it! 클린 프로그래밍 발표 자료

목차

Contents

Chapter 04

코드 리뷰 이해하기

4-1 코드 리뷰가 필요한 이유

4-2 코드 리뷰를 의미 있게 유지하기 어려운 이유

Chapter 05

코드 리뷰를 잘 하는 방법

5-1 클린하게 코드 리뷰 요청하기

5-2 효과적인 코드 리뷰 수행하기

5-3 코드 리뷰에 유용한 깃허브의 기능

APPENDIX 참고 자료: 자주 쓰이는 코드 리뷰 약어 (LGTM, PTAL, WDYT 등)

04

Chapter 04

코드 리뷰 이해하기

개발자와 리뷰어 간의 대화, 그 본질을 들여다보다

왜 코드 리뷰가 필요할까?

바쁜 업무 속에서도 코드 리뷰 문화가 실질적인 도움을 주는 이유

장점

- 코드 기능 수행 여부 확인 → 정확성 향상
- 변경된 코드를 동료가 쉽게 이해함
- 코드 베이스가 일관되게 관리됨
- 동료와 지식을 공유할 수 있음
- 프로젝트 히스토리를 저장해 추후 활용



기대 효과

- ★ 버그를 조기에 발견 → 시스템 장애 예방
- ★ 중복 코드 방지 · 모듈 재사용성 증가로 코드 품질 향상
- ★ 지식 공유로 개인과 팀의 역량 증대

가장 강조하고 싶은 가치

소스 코드의 공동화

한 개발자가 작성한 개별 코드가 코드 리뷰 과정을 거치면서
프로젝트에 참여한 모든 사람이 함께 수정·발전시켜 나가는 과정

Before. "내" 코드

- ✗ 내가 만든 코드이니 "내 것"으로 집착
- ✗ 혼자 만든 코드는 완벽하다고 주장하기 어려움
- ✗ 버그 발견·개선 기회 부족

After. "우리" 코드

- ✓ 협업으로 만든 공동 소유물이라는 인식
- ✓ 다른 개발자가 미처 못 본 버그 발견 가능
- ✓ 모두가 결과물에 주인 의식·책임감을 가짐

의미 있는 코드 리뷰, 왜 어려울까?

코드 리뷰는 **친절**, **예의**, **매너**가 강조되는 "대화"입니다.

왜? 개발자와 리뷰어의 입장이 종종 대립하기 때문

개발자 (리뷰 요청자)

💭 "자식 같은 '내' 코드니까 수정할 게 없을 거야"

- 코드에 큰 애정을 가짐
- 옳은 지적도 "기분 상함"으로 느끼기 쉬움
- 코드 비판 = 자기 비판처럼 받아들임

리뷰어

💭 "리뷰가 끝나면 '우리' 코드니까 꼼꼼하게 봐야지"

- 향후 문제가 생기지 않도록 책임감 가짐
- 냉철·비판적 시각으로 코드 검토
- "우리 프로젝트" 관점에서 접근

05

Chapter 05

코드 리뷰를 잘 하는 방법

리뷰 요청 단계부터 신중하게, 수행은 효과적으로

PR과 커밋 메시지는 "의미 있게" 작성

리뷰어가 읽기 쉽고 이해하기 편한 형태로 요청해야 합니다

 책 쓰기에 비유한다면?

PR = 책 제목과 소개 | 커밋 = 목차 | 리뷰 대상 코드 = 본문



리뷰하기 좋은 크기로 PR 생성

- PR이 크면 리뷰어에게 부담
- 세부 검토 어려워 리뷰 질 ↓
- 깃허브 우측 상단의 +/- 라인 수로 크기 확인



리뷰에 적극적으로 반응

- 반영 시: 처리 결과를 명확히 전달
- 미반영 시: **이유를 설명**하는 피드백 필수
- 무반응은 리뷰어 동기 저하 + 오해 유발

효과적인 리뷰 4가지 방법 (1/2)

기준: 리뷰를 요청한 개발자에게 얼마나 도움이 되었는가?

01

METHOD

친절하게 리뷰 수행하기

- 문자 기반 소통이라 **오해 소지가 큼** (목소리 톤·표정·제스처 없음)
- **부드러운 문체** + **청유형 문장** + **이모티콘 활용**을 권장

02

METHOD

레벨을 표시해 의견 제시하기

친절하게 작성하다 보면 의견이 모호해질 수 있음 → **레벨로 구분**

Comment 일반 리뷰 (승인·수정 요청 없음) · **Approve** 머지해도 좋음 ·
Request changes 수정 전까지 머지 금지

효과적인 리뷰 4가지 방법 (2/2)

03

METHOD

참고 자료 · 해결 방안과 함께 의견 제시

- ✓ **공신력 있는 참고 자료** → 개발자가 부족했던 부분을 쉽게 인식·개선
- ✓ **구체적인 해결 방법** 제시 → 수정 작업이 훨씬 수월해짐
- ✓ 리뷰 과정 자체가 개발자의 역량 향상에 기여

04

METHOD

구체적인 칭찬은 개발자를 춤추게 한다

- ✓ "무엇이 좋았는지, 어떤 점이 뛰어났는지"를 **명확히 언급**
- ✓ 공수는 적지만 **파급 효과는 매우 큼**
- ✓ 리뷰 받을 때 "평가받는다"는 부담을 줄여 → 효과적 리뷰의 원동력

자동으로 리뷰어 지정하기

CODEOWNERS 파일로 PR 생성 시 리뷰어가 자동 할당됩니다

사용 방법

1

파일 위치 선택

최상위 / docs/ / .github/

2

리뷰 대상·소유자 정의

패턴별로 담당자 지정

3

PR 생성

지정된 소유자가 자동으로 리뷰어 할당
(요청자 본인은 제외)

CODEOWNERS 작성 예시

```
CODEOWNERS

# 모든 파일 → @test 소유
* @test

# .java 파일 → 2인 공동 소유
*.java @test @test2

# 특정 경로 지정 + 하위 제외
/apps/ @test
/apps/github
```

최소 리뷰 승인 수 설정하기

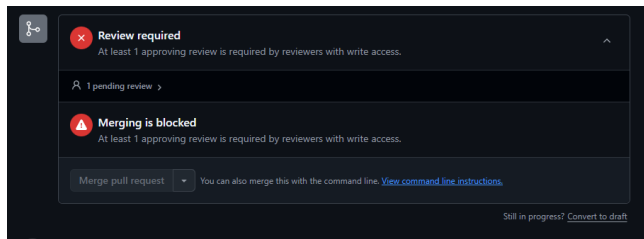
리뷰 없이 머지되는 위험한 PR을 막아 코드 리뷰를 "필수화"합니다

⚠ 설정하지 않으면

- 리뷰 없이도 PR 머지 가능
- 검증 없이 운영에 반영 → **시스템 장애 위험 ↑**

✓ 설정한 뒤

- 최소 승인 수 미달 시 머지 버튼 비활성화
- 코드 리뷰 수행을 **강제로 유도**



설정 절차



자주 쓰이는 코드 리뷰 약어

REFERENCE

책 외 / 현업·오픈소스에서 자주 쓰이는 표현을 정리한 자료입니다

LGTM

Looks Good To Me

좋아 보입니다 (승인)

PTAL

Please Take A Look

한번 봐주세요

WDYT

What Do You Think?

어떻게 생각하세요?

IMO

In My Opinion

제 생각에는

WIP

Work In Progress

작업 진행 중

TBD

To Be Determined

추후 결정 (미정)

FYI

For Your Information

참고로 알려드립니다

nit

nitpick

사소한 지적 (필수 ✕)

💡 예시 "[nit] 변수명을 camelCase로 바꾸면 좋을 것 같아요. WDYT?" → "LGTM! 머지하겠습니다 📌"

정리. 좋은 코드 리뷰란?

"코드 리뷰는 대화이고, 협업이며,
공동 소유물을 함께 발전시키는 과정이다."

요청자

PR과 커밋을
의미 있게 작성
+ 적극적 반응

리뷰어

친절·레벨·근거·
구체적 칭찬
4가지 기준 준수

팀 차원

CODEOWNERS와
최소 승인 수로
문화 정착